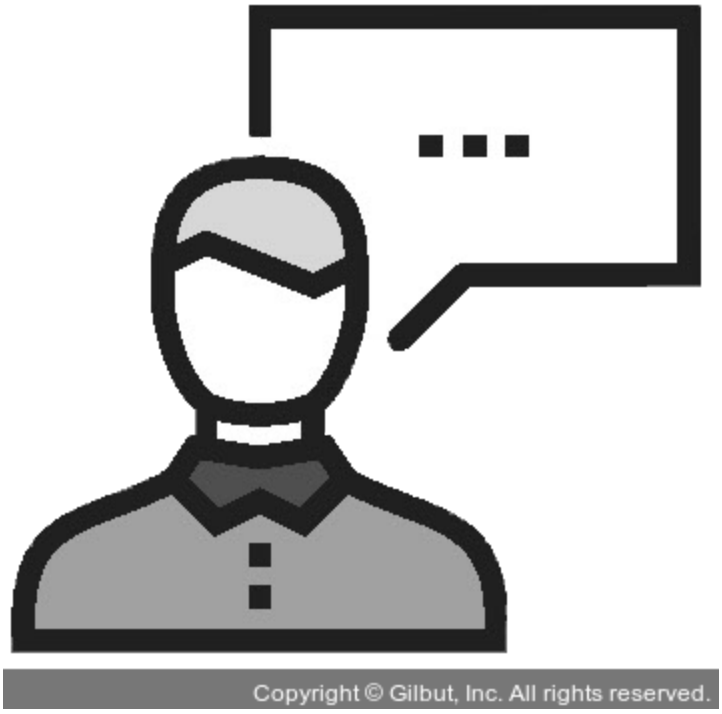
부록

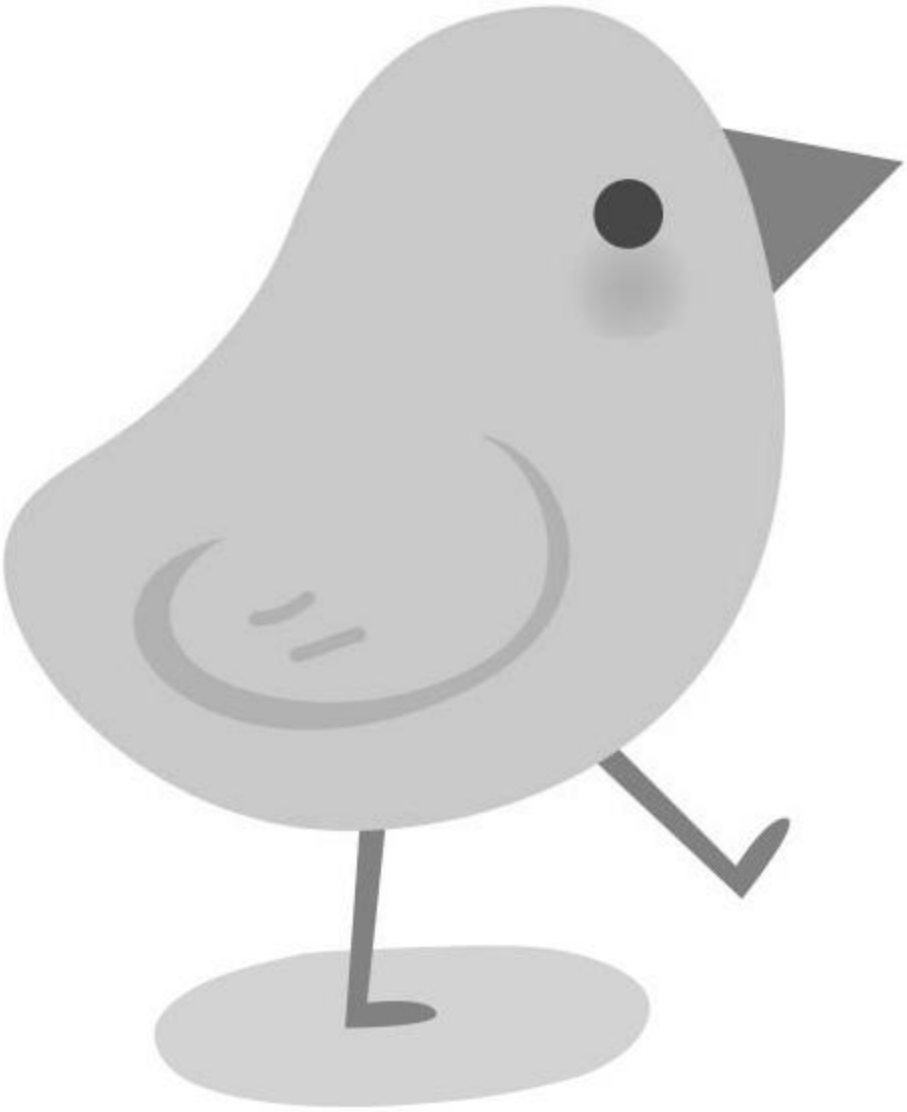
파이썬 병아리반



부록에서는 파이썬 기초를 빠르게 훑어 보고 지나가겠습니다. 따라서 개념 부분에 대한 설명은 자세하게 하지 않고, 코드를 중심으로 설명합니다. 이 책에서 다루는 기초 문법(입력, 출력, 연산자, `for` 반복문, `if` 조건문, 리스트 등)을 가볍게 복습하는 차원에서 또는 파이썬을 처음 배우는 사람들이 맛보기로 읽어도 좋습니다.

만약 다음 코드가 어렵지 않은 사람은 부록을 따로 읽지 않아도 괜찮습니다.

```
names = ['쵸파', '루피', '상디', '조로']
names.append('해적왕')
for name in names :
    if len(name) > 2 :
        print(name, '왔나요~?')
```



Copyright © Gilbut, Inc. All rights reserved.

1 출력과 입력 그리고 변수

지금부터 이 책에서 다뤄지는 파이썬의 가장 기초적인 내용을 살펴보려고 합니다. 먼저 여러분이 파이썬을 설치했다고 가정하고 진행하겠습니다.

■ print( ) 함수로 출력하기

여러분은 아마 print라는 단어가 친숙할 겁니다. 흔히 '프린트 했어?' 이런 표현들을 일상 생활에서도 많이 사용하는데요.

파이썬에서 `print`라는 이름의 함수는 괄호 안의 어떤 값을 모니터에 출력할 때 사용합니다. 백문이 불여일타(百聞而 不如一打)라고 했으니 먼저 코드를 작성해 보겠습니다. 자, `print`는 괄호 안의 어떤 값을 모니터에 출력해줄 때 사용한다고 했는데요, 파이썬을 처음 배우더라도 이 코드가 무엇을 의미하는지 짐작해 보세요.

```
print(34759**24341)
```

어떤 코드처럼 보이나요?



음..곱하기가 두 개니까.. 혹시 제곱하는 코드인가요?

■ 변수

변하는 값, 즉 변수에 대해 알아보겠습니다. 여러분도 웹 사이트에 로그인했을 때 이런 글귀를 본 적이 있을 겁니다.

파이쏭님! 안녕하세요!

그런데 이 글귀에서 어떤 부분은 사람에 따라 바뀌고, 어떤 부분은 사람에 관계없이 바뀌지 않는다는 것을 쉽게 알 수 있습니다. 어디가 바뀌는 부분인가요?



'파이쏭'이라는 닉네임 부분이 바뀌겠네요.

네, 맞습니다. 컴퓨터에서는 데이터를 저장하기 위해 공간이 필요합니다. 닉네임 부분처럼 하나의 값으로 고정되어 있지 않고 변하는 값을 저장할 수 있는 공간을 '변수(變數, variable)'라고 합니다.

변수는 다음과 같이 사용할 수 있습니다.

```
name = '파이쏭'
print(name)
```

■ input( ) 함수로 문자열 값 입력받기

다음 코드를 실행해보세요.

```
name = input()
print(name+'님! 안녕하세요!')
```

실행했는데 아무런 일이 일어나지 않아서 당황했나요? 여러분의 이름을 키보드로 입력하세요.

실행 결과

```
파이쏭—▶ 파이쏭 또는 여러분의 이름을 입력하세요!
파이쏭님! 안녕하세요!
```

앞에서 `print()` 함수는 괄호 안의 값을 모니터에 출력해주는 역할을 하는 함수라고 했었죠?

2 연산자 사용하기

연산자(演算子, operator)라는 말을 처음 들으면 무척 낯설게 느껴질 텐데요. 사실 우리는 연산자를 본 적이 있습니다. 그것도 4개나 말이죠. +, -, `**`, 그리고 오른쪽 값을 왼쪽 변수에 대입하는 역할을 했던 =도 연산자입니다. 코드를 차근차근 따라해보며 값을 이리저리 바꾸다 보면 금세 익숙해질 겁니다.

■ 산술 연산자(arithmetic operator)

첫 번째 만날 연산자는 산술(算術, arithmetic) 연산자입니다. 쉽게 말해서 숫자 계산을 위한 기호라고 생각하면 됩니다. 그리고 딱 이만큼만 알면 됩니다. 값을 이렇게 저렇게 바꿔보고, 여기에 없지만 앞에서 봤던 +, - 기호를 활용해서 계산을 해보기 바랍니다.

```
# 다양한 값을 넣어보며 연습해보세요.
print(3 * 10)   # * : 곱셈 연산자
print(3 ** 10)  # ** : 거듭 제곱 연산자
print(3 % 10)   # % : 나머지 연산자
print(3 / 2)    # / : 실수 나눗셈
print(3 // 2)   # // : 정수 나눗셈
```

실행 결과

```
13
59049
3
1.5
1
```

모든 연산자를 외울 필요는 없습니다. 필요한 상황에서 적절히 골라 사용할 수 있도록 어떤 연산자가 있는지만 알면 됩니다. 더 이상의 설명은 생략하겠습니다.

■ 비교 연산자(comparison operator)

두 번째 만날 연산자는 비교 연산자입니다. 이것도 코드를 보면 직관적으로 쉽게 이해가 갈 겁니다. 그리고 비교 연산자를 활용한 식이 참이면 True, 거짓이면 False라는 값을 갖는다는 것을 기억하세요.

```
# 다양한 값을 넣어보며 연습해보세요.
print(10 >= 3)
print(10 <= 3)
print(10 == 3)   # == : 같다
print(10 != 3)   # != : 같지 않다
print(3 % 2 == 1)
```

실행 결과

```
True
False
False
True
True
```

■ 논리 연산자(logical operator)

어렵게 느껴지던 연산자도 벌써 마지막 논리 연산자만을 남겨두고 있습니다. 논리 연산자는 앞에서 보았던 참과 거짓. 즉, True와 False와 관련된 연산입니다. 여기에서 다루지 않는 논리 연산자도 있지만, 병아리반에서는 다음의 and, or, not 3가지 연산자만 알면 충분합니다.

먼저 논리곱은 A and B와 같은 형태로 표현합니다. 다음 표에서 보는 것처럼 A와 B 중 하나라도 False일 경우에 결과는 False이고 둘 다 True일 때만 결과가 True가 됩니다.

논리식 (A and B)	결과
True and True	True
True and False	False
False and True	False
False and False	False

표 부록-1 논리곱 진리표

```
# 다양한 값을 넣어보며 연습해보세요.
print(1 == 1 and 2 != 1)          # True and True
print(10 % 2 != 0 and 1 + 1 > 0)  # True and False
```

실행 결과

```
False
True
```

3 함수 불러오기

지금까지 우리가 사용했던 함수에는 어떤 것들이 있었나요?



`print`, `input`, `int`, `str`이요!

지금까지 사용했던 함수들은 파이썬에서 항상 사용할 수 있도록 제공하는 내장 함수(Built-in Functions)입니다. 현재 파이썬 3.7 버전에서 제공하는 내장 함수의 종류는 다음과 같습니다. 이중 몇 가지는 앞으로 사용하겠지만, 모두 알 필요는 전혀 없습니다.

내장 함수				
abs()	delattr()	hash()	memoryview()	set()
all()	dict()	help()	min()	setattr()
any()	dir()	hex()	next()	slice()
ascii()	divmod()	id()	object()	sorted()
bin()	enumerate()	input()	oct()	staticmethod()
bool()	eval()	int()	open()	str()
breakpoint()	exec()	isinstance()	ord()	sum()
bytearray()	filter()	issubclass()	pow()	super()
bytes()	float()	iter()	print()	tuple()
callable()	format()	len()	property()	type()
chr()	frozenset()	list()	range()	vars()
classmethod()	getattr()	locals()	repr()	zip()

compile()	globals()	map()	reversed()	__import__()
complex()	hasattr()	max()	round()	

표 부록-4 파이썬 내장 함수

4 반복과 선택

이제 가장 중요한 부분입니다. 컴퓨터는 사람보다 빠르게 많은 양의 데이터를 처리할 수 있습니다. 이때 반복과 선택을 통해 정확하고 효율적인 동작이 이루어질 수 있도록 프로그래밍할 수 있습니다.

■ for 반복문

먼저 명령어를 반복하여 실행시키는 방법을 알아봅시다. 반복문을 적절히 사용하면 단 몇 줄의 코드로도 원하는 결과를 얻을 수 있습니다.

파이썬에서 for 반복문은 주어진 데이터 세트를 순회하거나 원하는 횟수만큼 반복하고 싶을 때 사용합니다.

`for` 반복문의 구조는 다음과 같습니다.

```
for A in B :          # 맨 뒤에 콜론 기호(:)를 잊지 마세요.
    반복할 문장1      # 들여쓰기(문단 구분)가 되어 있음
    (반복할 문장n)    # 보통 공백 문자([space]) 4개로 사용함
```

여기에서 B는 데이터 세트를 의미합니다. 그리고 데이터 세트에서 데이터를 하나씩 꺼내 와서 A에 임시로 저장합니다. 그리고 데이터 세트 뒤에 ':'을 쓰고 그 밑에 반복할 문장을 작성합니다. 여기에서 반복할 문장은 문단 구분 같은 것이 되어있는 것을 볼 수 있는데요. 바로 파이썬을 처음 배울 때 가장 성가신 점인 들여쓰기(indent) 규칙입니다. 자동으로 들여쓰기가 되는 경우도 있지만 직접 들여쓰기를 해야할 경우에는 공백 문자, 즉 Space를 4번 치면 됩니다.

■ if 조건문

우리는 매일 매일 수많은 선택을 하며 살아가는데요. 컴퓨터가 조건에 맞게 명령어를 선택하도록 조건문을 사용할 수 있습니다. 프로그래밍에서 선택은 반복과 함께 가장 중요한 개념입니다.

파이썬을 포함한 대부분의 프로그래밍 언어에서 `if` 조건문은 주어진 조건이 True(참)인 경우에만 명령을 선택적으로 실행하고 싶을 때 사용합니다.

`if` 조건문의 구조는 다음과 같습니다. `for` 반복문과 마찬가지로 콜론 기호(:)를 사용하고 조건이 참일 때 실행될 문장은 4칸 들여쓰기를 해야 합니다.

```
if 조건 :                          # 맨 뒤에 ':' 콜론을 잊지 마세요!
    조건이 참일 경우 실행할 문장1    # 들여쓰기가 되어 있음
    (조건이 참일 경우 실행할 문장1)  # 들여쓰기를 잊지 마세요!
```

코드를 보면 더 쉽게 이해할 수 있습니다. 다음 코드에서 `if` 뒤의 조건은 참인가요?

```
if 10 > 0 :
    print('안녕하세요?')
```



네, 10은 0보다 크다는 조건은 True(참)입니다. 그러면 그 다음 들여쓰기 된 `print()` 함수가 실행되는 건가요?

5 순서 있는 저장 공간 리스트

드디어 파이썬 병아리반의 마지막 순서입니다. 우리 주변에 순서가 있는 데이터 종류로는 어떤 것들이 있을까요? 음식점에서 대기 명단에 이름을 적는 것에도 순서가 있고, 학번에도 순서가 있습니다.

파이썬에서는 순서가 있는 데이터를 다룰 때, 리스트(List) 데이터 구조를 사용합니다.

사실 우리는 리스트라는 것을 이미 앞에서 본 적이 있습니다. 바로 이 코드들에 있는 대괄호로 감싼 부분이 바로 리스트입니다.

```
for i in [0,1,2,3] :
    print(i ** 2)
names = ['쵸파','루피','상디','조로']
for name in names :
    print(name)
```

여기에서 알 수 있듯이 리스트는 전체 데이터를 대괄호로 감싸주고 각각의 값들은 콤마로 구분합니다.

■ 리스트에 저장된 위치(index)로 값에 접근하기(indexing)

여러분은 `range()` 함수가 0부터 n-1까지의 연속된 정수를 생성했던 것을 기억할 겁니다. 프로그래밍을 하다 보면 1부터 숫자를 세는 것이 익숙한 사람과 달리 컴퓨터는 0부터 숫자를 세는 경우를 많이 볼 수 있는데요. 그러면 이 코드의 실행 결과는 무엇일까요?

```
names = ['쵸파','루피','상디','조로']
print(names[1])
```



`[1]`이라는 코드가 무엇을 의미하는지는 잘 모르겠지만, 컴퓨터는 보통 0부터 센다고 했으니 맨 앞에 있는 '쵸파'가 0번일 것 같네요. 그러면 '루피'가 출력되나요?

아주 훌륭합니다. 그러면 대괄호 안의 숫자가 무엇을 의미하는지 이해했나요?



음...리스트에 저장된 데이터의 위치인 것 같아요!

■ 리스트에 저장된 위치로 데이터의 일부 자르기(slicing)

리스트의 또 다른 특성은 순서에 따라 여러 데이터에 접근할 수 있다는 것입니다. 이것을 슬라이스(slice)라고 하는데요. 다음 코드처럼 사용할 수 있습니다.

```
names = ['쵸파','루피','상디','조로']
print(names[0:2])
print(names[1:3])
print(names[1:])
print(names[:])
```

실행 결과

```
['쵸파', '루피']
['루피', '상디']
['루피', '상디', '조로']
['쵸파', '루피', '상디', '조로']
```

슬라이스를 할 때도 `range()` 함수와 마찬가지로 `[a:b]`는 `a` 이상 `b` 미만이라는 구간이 적용되는 것을 볼 수 있으며, 생략할 경우 '맨 앞에서부터' 또는 '끝까지'와 같은 기능을 하는 것을 볼 수 있습니다.

■ 리스트에 값 추가하기

이제 주어진 리스트에 값을 추가하는 방법을 알아보겠습니다. `append`라는 단어를 사전에서 찾아보면 '덧붙이다', '첨부하다'라는 뜻입니다. 그래서 리스트에 값을 추가할 때는 다음과 같이 `append`라는 함수를 사용하면 됩니다.

```
names = ['쵸파','루피','상디','조로']
names.append('나미')  # 리스트에 값 추가하기
print(names)
```

실행 결과

```
['쵸파', '루피', '상디', '조로', '나미']
```

그리고 리스트에 몇 개의 데이터가 저장되었는지 확인하려면 `length`라는 뜻의 `len()` 함수를 사용하면 됩니다. `len()` 함수는 꼭 리스트의 길이만을 알 수 있는 것은 아니고, 문자열의 길이도 알 수 있습니다.

```
print(len(names))                        # 리스트 길이 출력하기
print(len('data analysis for everyone'))  # 문자열 길이 출력하기
```

실행 결과

```
5
26
```